

TelemetriaMqtt v2 — Checkpoint 1

Scaffolding do repositório: firmware PlatformIO, template de credenciais, harness de testes, publicação no GitHub e CI

Projeto de Telemetria FSAE

16 de setembro de 2026 (atualizado)

Abstract

Este documento detalha o primeiro checkpoint do projeto de reconstrução do sistema de telemetria da equipe: a criação da estrutura inicial do repositório, com foco no firmware do ESP32. O checkpoint estabelece o projeto PlatformIO com dois ambientes de build (um para o hardware real, outro para testes sem hardware), o template de credenciais que nunca é versionado, e a validação de que toda a cadeia de ferramentas (compilação e testes) funciona de ponta a ponta antes de qualquer lógica de negócio ser escrita. Inclui também, como adendo (ver Seção 5), a publicação do repositório no GitHub e a configuração de integração contínua (CI) — trabalho que, em retrospecto, pertence ao escopo de scaffolding deste checkpoint, mas que só foi executado depois do checkpoint 4, ao ser identificada essa lacuna.

Contents

1	Contexto e objetivo	2
2	Por que PlatformIO em vez da IDE Arduino	2
2.1	Instalação	2
3	Estrutura de arquivos criada	2
3.1	platformio.ini	3
3.2	src/main.cpp	3
3.3	include/secrets.h.example	3
3.4	test/test_scaffolding/test_main.cpp	4
3.5	.gitignore	4
4	Validação: rodando de verdade	5
4.1	Build do ambiente de produção	5
4.2	Testes do ambiente nativo	5
4.3	Verificação do .gitignore	5
5	Adendo: publicação no GitHub, CI e releases	6
5.1	Por que isso deveria ter sido feito no checkpoint 1	6
5.2	O que foi feito	6
5.3	Validação: rodando de verdade	6
6	Resumo do que ficou pronto	7
7	Próximos passos	7

1 Contexto e objetivo

Este é o primeiro checkpoint de um projeto reconstruído do zero a partir do TCC anterior da equipe (repositório antigo preservado, sem reaproveitamento de código, em `~/Desktop/TelemetriaMqtt`). Antes de escrever qualquer lógica — leitura de CAN, publicação MQTT, etc. — o objetivo deste checkpoint é puramente de **scaffolding**: montar o esqueleto do repositório de forma que exista, desde o primeiro commit, uma forma de compilar o firmware e uma forma de rodar testes automatizados, seguindo a metodologia de TDD definida para o projeto (documentada em `CLAUDE.md`).

A decisão de arquitetura tomada antes deste checkpoint (registrada em `SCOPE.md`) foi usar um único ESP32 (em vez da dupla Arduino Uno + ESP32 do projeto anterior) com PlatformIO como ferramenta de build, em vez da IDE Arduino tradicional.

2 Por que PlatformIO em vez da IDE Arduino

A IDE Arduino tradicional (`.ino`, compilação feita clicando em um botão) não oferece de forma nativa:

- Múltiplos ambientes de build a partir do mesmo código-fonte (por exemplo, um ambiente para o hardware real e outro só para rodar testes no computador).
- Um framework de testes unitários integrado.
- Compilação via linha de comando, necessária para automatizar validações (rodar o build como parte do fluxo de trabalho, sem depender de abrir uma janela gráfica).

O PlatformIO resolve os três pontos através do arquivo `platformio.ini`, que declara ambientes (*environments*) nomeados, cada um com sua própria plataforma de hardware, framework e conjunto de bibliotecas.

2.1 Instalação

O PlatformIO não estava pré-instalado no ambiente de desenvolvimento. Foi instalado via `pip`, isolado no diretório do usuário:

Listing 1: Instalação do PlatformIO

```
1 pip3 install --user platformio
```

Isso adiciona o executável `pio` a `~/local/bin`, sem exigir privilégios de administrador nem afetar o Python do sistema.

3 Estrutura de arquivos criada

Listing 2: Estrutura do diretório firmware/

```
1 firmware/  
2   platformio.ini  
3   src/  
4     main.cpp  
5   include/  
6     secrets.h.example  
7   test/  
8     test_scaffolding/  
9       test_main.cpp
```

3.1 platformio.ini

Listing 3: firmware/platformio.ini

```
1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = arduino
5 monitor_speed = 115200
6
7 [env:native]
8 platform = native
9 test_framework = unity
10 build_flags = -std=c++17
```

Dois ambientes com propósitos completamente diferentes:

- **esp32dev** — ambiente de produção. Usa o framework Arduino sobre a plataforma **espressif32** (fabricante do ESP32), gerando um binário que só roda no hardware real.
- **native** — ambiente de testes. Compila e executa **no próprio computador de desenvolvimento**, usando o compilador nativo do sistema operacional em vez de um cross-compiler para o ESP32. Usa o framework de testes *Unity* (biblioteca padrão de testes em C para ambientes embarcados). A partir do checkpoint 8, quando o parser do protocolo CAN for implementado, esse ambiente permitirá testar a lógica de decodificação sem precisar de um ESP32 conectado por USB a cada execução de teste.

Essa separação é o que possibilita cumprir o requisito de TDD do projeto mesmo em código embarcado: lógica pura (parsing, cálculos, formatação) pode ser testada no ambiente **native**, rápido e sem hardware; só o código que de fato depende de periféricos físicos (leitura do barramento CAN, WiFi) precisa ser validado no hardware real.

3.2 src/main.cpp

Listing 4: firmware/src/main.cpp

```
1 #include <Arduino.h>
2
3 void setup() {
4     Serial.begin(115200);
5     Serial.println("TelemetriaMqtt v2 - firmware placeholder");
6 }
7
8 void loop() {
9 }
```

Um placeholder mínimo, deliberadamente sem nenhuma lógica — serve apenas para confirmar que o ambiente **esp32dev** compila com sucesso antes de qualquer funcionalidade ser adicionada nos próximos checkpoints (8 e 9).

3.3 include/secrets.h.example

Listing 5: firmware/include/secrets.h.example

```
1 #pragma once
2
3 // Copie este arquivo para secrets.h (gitignored) e preencha com
4 // valores reais. Nunca commitar secrets.h.
5
6 #define WIFI_SSID "SSID_AQUI"
7 #define WIFI_PASSWORD "SENHA_AQUI"
```

```

8
9 #define MQTT_BROKER_HOST "seu-broker.exemplo.com"
10 #define MQTT_BROKER_PORT 8883
11 #define MQTT_USERNAME "usuario-do-dispositivo"
12 #define MQTT_PASSWORD "senha-do-dispositivo"
13 #define MQTT_TOPIC "telemetria/esp32"

```

Esse arquivo existe por uma razão concreta identificada durante o levantamento de requisitos do projeto: o repositório do TCC anterior tinha credenciais reais (senha de WiFi e credenciais do broker MQTT) escritas diretamente no código-fonte, e esse repositório foi publicado publicamente no GitHub — expondo essas credenciais. Para não repetir o problema, o padrão adotado desde o primeiro commit deste projeto é: um arquivo `.example` versionado, servindo de modelo, e o arquivo real (`secrets.h`, sem o sufixo) listado no `.gitignore`, nunca chegando a existir na história do repositório.

3.4 test/test_scaffolding/test_main.cpp

Listing 6: firmware/test/test_scaffolding/test_main.cpp

```

1 #include <unity.h>
2
3 // Prova de que o harness de testes nativo (sem hardware) está
4 // funcionando. Passa a valer pra lógica de verdade a partir do
5 // checkpoint 8 (parser CAN).
6 void test_native_test_harness_runs(void) {
7     TEST_ASSERT_EQUAL(4, 2 + 2);
8 }
9
10 int main(int argc, char **argv) {
11     UNITY_BEGIN();
12     RUN_TEST(test_native_test_harness_runs);
13     return UNITY_END();
14 }

```

Um teste deliberadamente trivial ($2 + 2 == 4$). Seu único propósito não é validar lógica de negócio — que ainda não existe — mas provar que toda a cadeia de ferramentas do ambiente **native** funciona: o PlatformIO consegue baixar e linkar a biblioteca Unity, compilar um executável nativo, rodá-lo, e reportar corretamente um resultado **PASSED**. Detectar um problema de configuração aqui, com um teste trivial, é muito mais barato do que descobrir esse mesmo problema mais tarde, junto com a primeira lógica real a ser testada.

3.5 .gitignore

Listing 7: .gitignore (raiz do projeto)

```

1 # PlatformIO
2 firmware/.pio/
3 firmware/include/secrets.h
4
5 # Python
6 __pycache__/
7 *.pyc
8 .venv/
9 venv/
10
11 # Node
12 node_modules/
13
14 # OS / editor
15 .DS_Store

```

Além de `secrets.h`, o diretório `firmware/.pio/` (onde o PlatformIO baixa toolchains, bibliotecas e coloca os artefatos de build) também é ignorado — são vários gigabytes de arquivos gerados automaticamente, que não fazem sentido versionar.

4 Validação: rodando de verdade

Consistente com a metodologia do projeto (nenhum checkpoint é considerado fechado só por os arquivos existirem — é preciso rodar e confirmar o resultado), os dois ambientes foram executados e o resultado observado diretamente, não apenas assumido.

4.1 Build do ambiente de produção

Listing 8: Saída de `pio run -e esp32dev` (resumida)

```

1 Compiling .pio/build/esp32dev/FrameworkArduino/...
2 Linking .pio/build/esp32dev/firmware.elf
3 Checking size .pio/build/esp32dev/firmware.elf
4 RAM:      [=          ]   6.6% (used 21464 bytes from 327680 bytes)
5 Flash:    [==          ]  20.4% (used 267057 bytes from 1310720 bytes)
6 Building .pio/build/esp32dev/firmware.bin
7 ===== [SUCCESS] Took 49.42 seconds
   =====

```

O binário foi gerado com sucesso, usando apenas 6,6% da RAM e 20,4% da memória flash disponíveis no ESP32 — números baixos porque o firmware ainda é só o placeholder, mas confirmam que a cadeia de compilação (download do framework Arduino para ESP32, compilador, linker, geração da imagem `.bin`) está correta e funcional.

4.2 Testes do ambiente nativo

Listing 9: Saída de `pio test -e native`

```

1 Processing test_scaffolding in native environment
2 Building...
3 Testing...
4 test/test_scaffolding/test_main.cpp:11: test_native_test_harness_runs [PASSED
  ]
5 ----- native:test_scaffolding [PASSED] Took 0.47 seconds
   -----
6 1 test cases: 1 succeeded in 0.475 seconds

```

O teste trivial passou, confirmando que o harness de testes (download da biblioteca Unity, compilação nativa, execução, relatório de resultado) está pronto para receber testes de lógica real nos próximos checkpoints.

4.3 Verificação do `.gitignore`

Como parte da mesma validação, foi confirmado que o `.gitignore` de fato exclui os arquivos sensíveis e gerados, usando o comando de diagnóstico do próprio Git:

Listing 10: Verificação do `.gitignore`

```

1 $ git check-ignore -v firmware/.pio/build firmware/include/secrets.h
2 .gitignore:2:firmware/.pio/                firmware/.pio/build
3 .gitignore:3:firmware/include/secrets.h    firmware/include/secrets.h

```

Ambos os caminhos são corretamente ignorados pela regra correspondente do arquivo.

5 Adendo: publicação no GitHub, CI e releases

Esta seção foi adicionada depois do checkpoint 4, quando ficou claro que scaffolding de repositório deveria ter incluído, desde o primeiro commit, um remote público, integração contínua e um esquema de releases — não só a estrutura de pastas e o build local. Documentado aqui, e não num documento à parte, porque conceitualmente **pertence ao escopo deste checkpoint**, mesmo tendo sido implementado depois. O `CLAUDE.md` foi atualizado para refletir esse escopo corrigido nos próximos projetos.

5.1 Por que isso deveria ter sido feito no checkpoint 1

Scaffolding de repositório não é só "o código compila localmente" — é toda a base operacional que os checkpoints seguintes vão depender. Deixar o remote e o CI para depois significa que boa parte do projeto (checkpoints 1 a 4) foi desenvolvida sem nenhuma rede de proteção automatizada: nenhum push disparava testes, não havia histórico de builds, e o repositório só existia numa única máquina. O trabalho de auditoria retroativa (verificar se os testes cobriam o que foi implementado, revisado separadamente) só foi necessário, em parte, porque não havia CI cobrando isso a cada commit desde o início.

5.2 O que foi feito

- **Publicação do repositório:** GitHub, público, em <https://github.com/ThallesNonato1123/TelemetriaMqtt-v2>. Antes do primeiro push, foi feita uma varredura em todo o histórico de commits por credenciais acidentalmente versionadas (`git log -p -all | grep ...`) — encontradas apenas senhas de desenvolvimento descartáveis em exemplos de comando na documentação, sem risco real, trocadas por um placeholder genérico por higiene.
- **Tags semânticas retroativas:** uma tag por checkpoint fechado (v0.1.0 a v0.4.1), apontando para os commits que efetivamente fecharam cada etapa.
- **CI (`.github/workflows/ci.yml`):** três jobs paralelos no GitHub Actions, um por suíte de teste independente do projeto (firmware nativo via PlatformIO, backend via pytest, infra via pytest subindo containers Mosquitto efêmeros), rodando a cada push e pull request.

Listing 11: Trecho de `.github/workflows/ci.yml`

```
1 jobs:
2   firmware:
3     runs-on: ubuntu-latest
4     steps:
5       - uses: actions/checkout@v4
6       - uses: actions/setup-python@v5
7       - run: pip install platformio
8       - working-directory: firmware
9         run: pio test -e native
10  backend: ...
11  infra: ...
```

Os runners `ubuntu-latest` do GitHub Actions já vêm com Docker instalado e o daemon ativo por padrão — por isso o job de `infra/` (que sobe containers Mosquitto reais durante os testes, como descrito no documento do checkpoint 3) funciona sem nenhuma configuração adicional.

5.3 Validação: rodando de verdade

Depois do primeiro push com o workflow de CI, a execução foi acompanhada em tempo real (`gh run watch`) até completar, confirmando que as três jobs passam de fato no ambiente do GitHub

Actions, não só localmente:

Listing 12: Resultado real do primeiro run de CI

```
1   Firmware (testes nativos, sem hardware) in 19s
2   Backend (mock publisher + ingestão) in 12s
3   Infra (Mosquitto ACL/auth, via Docker) in 21s
```

6 Resumo do que ficou pronto

- Projeto PlatformIO com dois ambientes: `esp32dev` (build real) e `native` (testes sem hardware).
- Firmware placeholder compilando com sucesso para o ESP32.
- Template de credenciais (`secrets.h.example`) versionado; o arquivo real nunca chega a ser rastreado pelo Git.
- Harness de testes unitários (Unity) funcionando ponta a ponta, pronto para receber testes de lógica real.
- `.gitignore` validado na prática, não apenas escrito.
- **(Adendo)** Repositório público no GitHub, com histórico auditado antes da publicação.
- **(Adendo)** CI rodando as três suítes de teste do projeto a cada push, validado com uma execução real.
- **(Adendo)** Esquema de releases (tag semântica por checkpoint) estabelecido e aplicado retroativamente.

7 Próximos passos

O checkpoint seguinte (checkpoint 2, conforme `CLAUDE.md`) muda de área: em vez do firmware, cria o script mock de publicação MQTT em Python, que simula o comportamento do ESP32 sem depender de nenhum hardware, permitindo que o desenvolvimento do restante do backend (broker, ingestão, dashboards) avance em paralelo à obtenção física do ESP32 e do transceiver CAN.